

Estudo Dirigido 03

Padronização em Banco de Dados

Administração de banco de dados

Integrantes:

- Adriano Antônio Nogueira de Paula
 - Bernardo Ladeira Leal de Medeiros
 - Igor Landim Oliveira
 - João Hugo Teixeira Martins Botelho
 - Mateus Silva Xavier
 - Pedro Paiva Ferreira
 - Pedro Sias Ferreira
-

1. Introdução

A padronização em bancos de dados é fundamental para garantir organização, integridade, segurança e facilidade de manutenção das informações armazenadas em um Sistema Gerenciador de Banco de Dados (SGBD). Em ambientes de desenvolvimento, a ausência de padrões pode ocasionar inconsistências estruturais, dificuldades de manutenção, redundância de dados e falhas de segurança.

O PostgreSQL é um dos SGBDs relacionais mais utilizados atualmente, oferecendo recursos avançados de integridade, controle de acesso, indexação e modelagem relacional. Dessa forma, a adoção de boas práticas de padronização nesse ambiente contribui diretamente para a confiabilidade e desempenho dos sistemas.

O presente trabalho apresenta uma análise prática de padronização aplicada a um banco de dados desenvolvido no PostgreSQL, abordando aspectos relacionados à nomenclatura, tipos de dados, integridade, normalização, segurança, índices e documentação.

2. Práticas de Padronização

2.1 Convenções de Nomenclatura

A padronização de nomenclaturas facilita a leitura, organização e manutenção do banco de dados. No sistema analisado, observou-se a utilização do padrão `_id` para identificação das chaves estrangeiras, como nas colunas **carro_id** e **acessorios_id** da tabela associativa `carro_acessorio`.

A Figura 1 demonstra a utilização desse padrão, evidenciando maior clareza na identificação dos relacionamentos entre tabelas.

Entretanto, observou-se inconsistência na coluna **acessorios_id**, nomeada no plural, enquanto as demais colunas seguem o padrão singular. Como boa prática, recomenda-se a utilização uniforme de nomes no singular, como **acessorio_id**, garantindo maior consistência estrutural.

Além disso, a Figura 2 apresenta as constraints de chave estrangeira geradas automaticamente pelo Hibernate. Os nomes produzidos automaticamente reduzem a legibilidade do banco de dados, sendo recomendável a utilização de nomenclaturas mais descritivas para constraints.

Figura 1 - Estrutura de colunas da tabela `carro_acessorio`

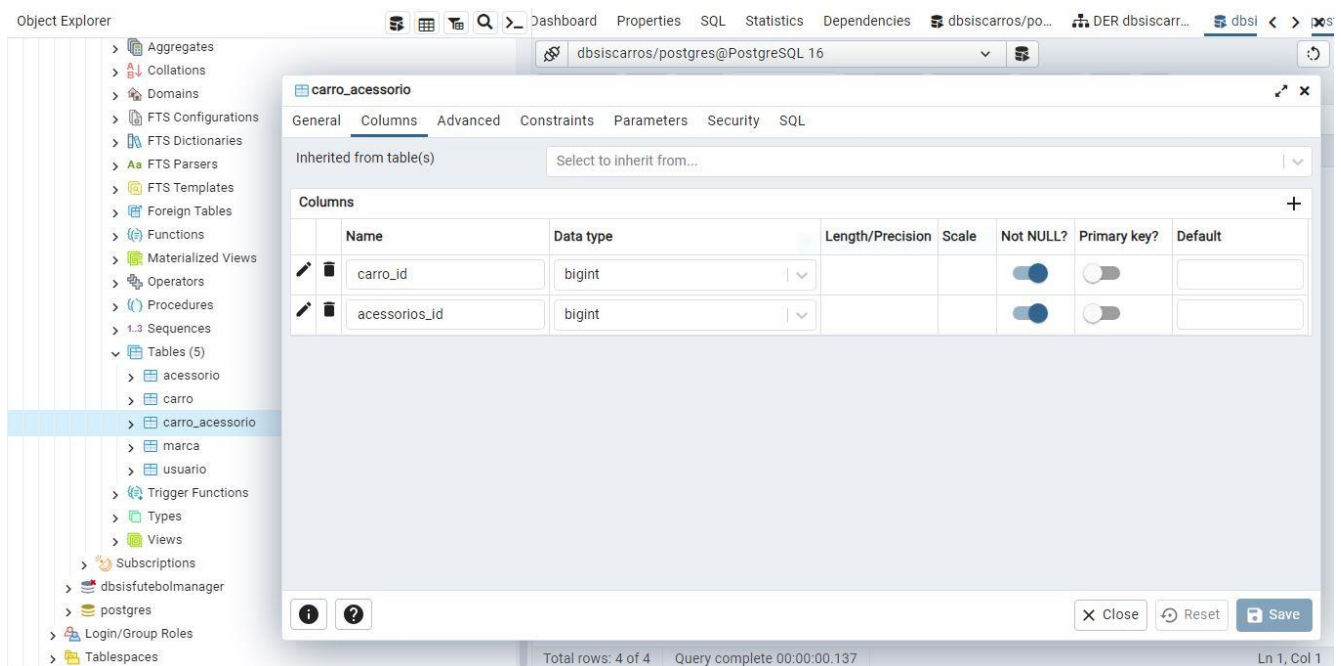
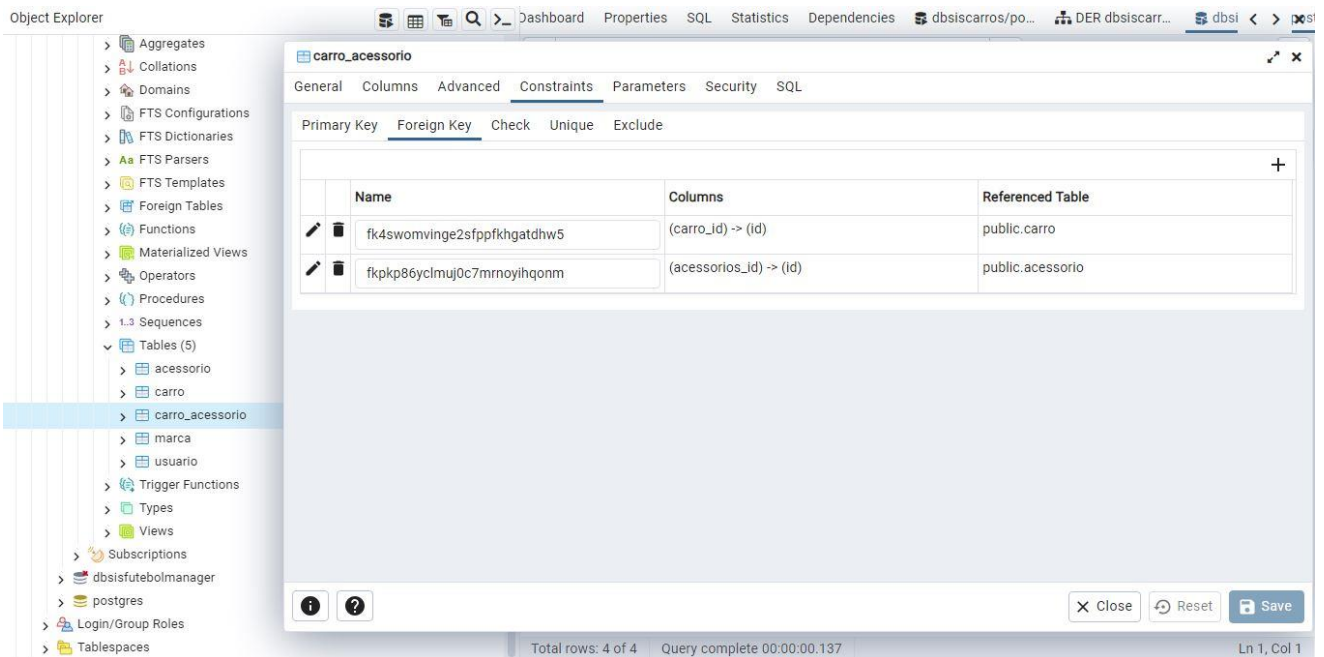


Figura 2 - Constraints da tabela carro_acessorio



2.2 Padronização de Tipos de Dados

2.2.1 Divergência entre aplicação e banco

Durante a análise do sistema, identificou-se divergência entre os tipos de dados definidos na aplicação Java e os tipos implementados no PostgreSQL. Na entidade Acessorio, **o atributo id foi definido como Integer**, enquanto no banco o campo correspondente foi criado como BIGINT.

Essa inconsistência evidencia a importância da compatibilidade entre aplicação e banco de dados, evitando possíveis problemas de integração e manutenção.

Para sistemas menores, recomenda-se a utilização de Integer no Java associado ao tipo INTEGER no PostgreSQL. Já em sistemas escaláveis, recomenda-se a utilização de Long juntamente com BIGINT.

Na IDE Eclipse Java

```
Acessorio.java x
13 @AllArgsConstructor
14 @NoArgsConstructor
15 @Entity
16 @Table(name = "acessorio")
17
18 public class Acessorio {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Integer id;
23
24     @Column(length = 20)
25     private String nome;
26
27     //Getters e Setters
28     public Integer getId() {
29         return id;
30     }
31 }
32
33
```

No Pg Admin

Object Explorer

- > Collations
- > Domains
- > FTS Configurations
- > FTS Dictionaries
- > FTS Parsers
- > FTS Templates
- > Foreign Tables
- > Functions
- > Materialized Views
- > Operators
- > Procedures
- > Sequences
- > Tables (5)
 - acessorio
 - carro
 - carro_acessorio
 - marca
 - usuario
- > Trigger Functions
- > Types
- > Views
- > Subscriptions
- > dbisifutebolmanager
- > postgres
- > Login/Group Roles
- > Tablespaces

dbisiscarras/postgres@PostgreSQL 16

acessorio

General Columns Advanced Constraints Parameters Security SQL

Inherited from table(s) Select to inherit from...

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id	bigint			☒	☒	
	nome	character varying	20		☐	☐	

Total rows: 4 of 4 Query complete 00:00:00.137 Ln 1, Col 1

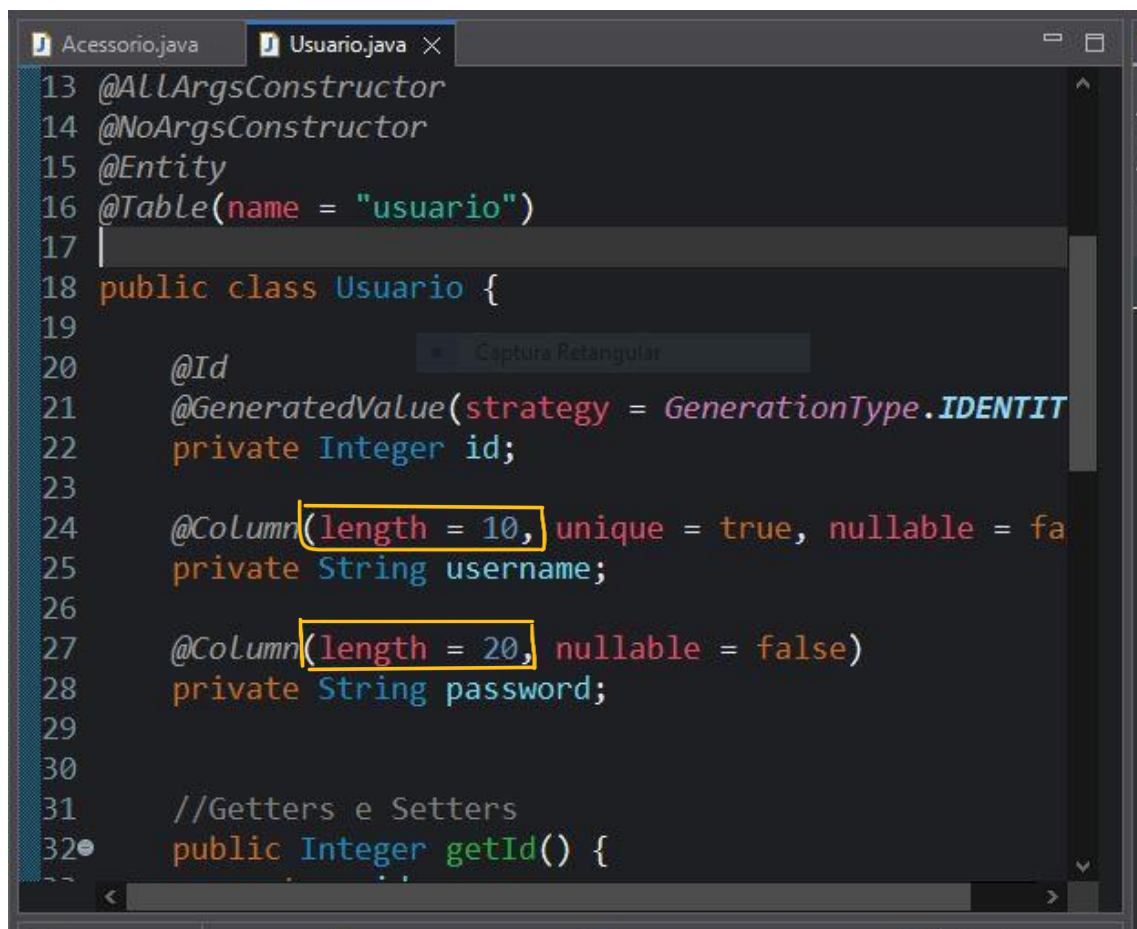
2.2.2 Campos muito pequenos

Observou-se também a utilização de campos textuais com tamanho reduzido, como username VARCHAR(10) e password VARCHAR(20).

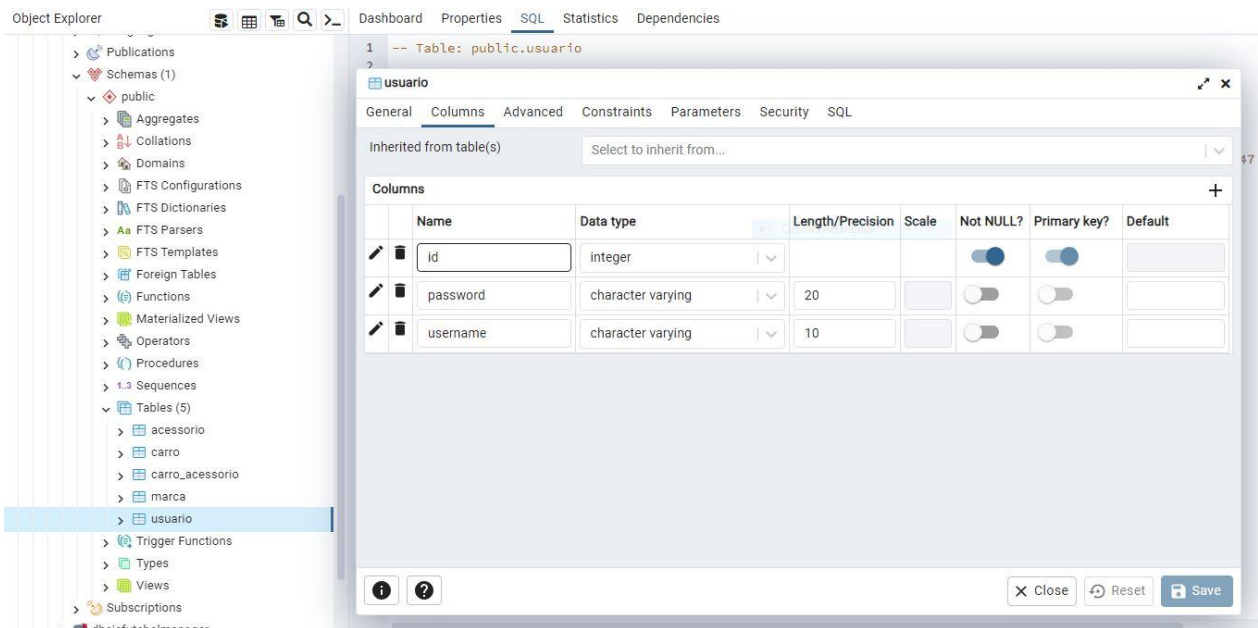
Embora o tamanho de username possa ser suficiente para sistemas pequenos, essa definição pode limitar a flexibilidade futura da aplicação.

Já o campo password VARCHAR(20) apresenta inadequação para aplicações modernas, especialmente em cenários que utilizam algoritmos de hash criptográfico, como bcrypt e Argon2, que normalmente exigem mais espaço de armazenamento.

Dessa forma, a padronização de tipos de dados deve considerar não apenas compatibilidade estrutural, mas também segurança, escalabilidade e manutenção futura.



```
13 @AllArgsConstructor
14 @NoArgsConstructor
15 @Entity
16 @Table(name = "usuario")
17
18 public class Usuario {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Integer id;
23
24     @Column(length = 10, unique = true, nullable = false)
25     private String username;
26
27     @Column(length = 20, nullable = false)
28     private String password;
29
30
31     //Getters e Setters
32     public Integer getId() {
```

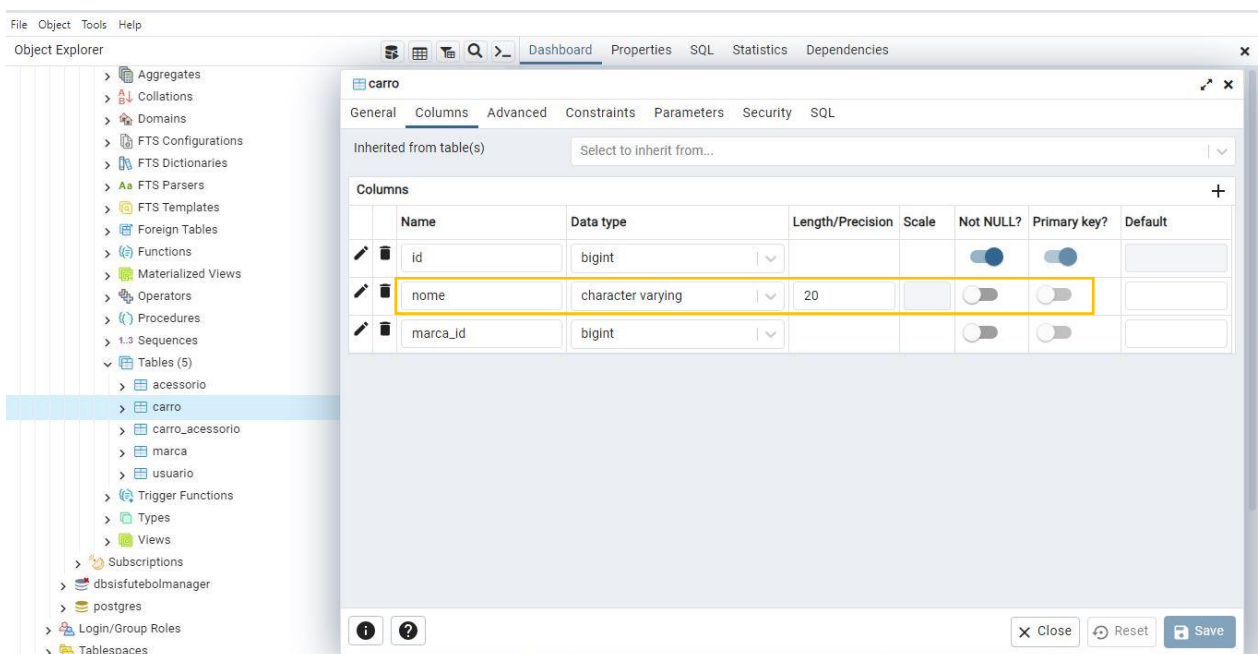


2.3 Integridade dos Dados

2.3.1 Chave estrangeira aceitando NULL

A análise da tabela carro demonstrou que a coluna **marca_id** permite valores NULL, possibilitando o cadastro de veículos sem associação a uma marca.

Essa situação pode comprometer a integridade referencial e gerar inconsistências nos relacionamentos do banco de dados. Recomenda-se a utilização da restrição NOT NULL para garantir que todo veículo esteja vinculado a uma marca válida.

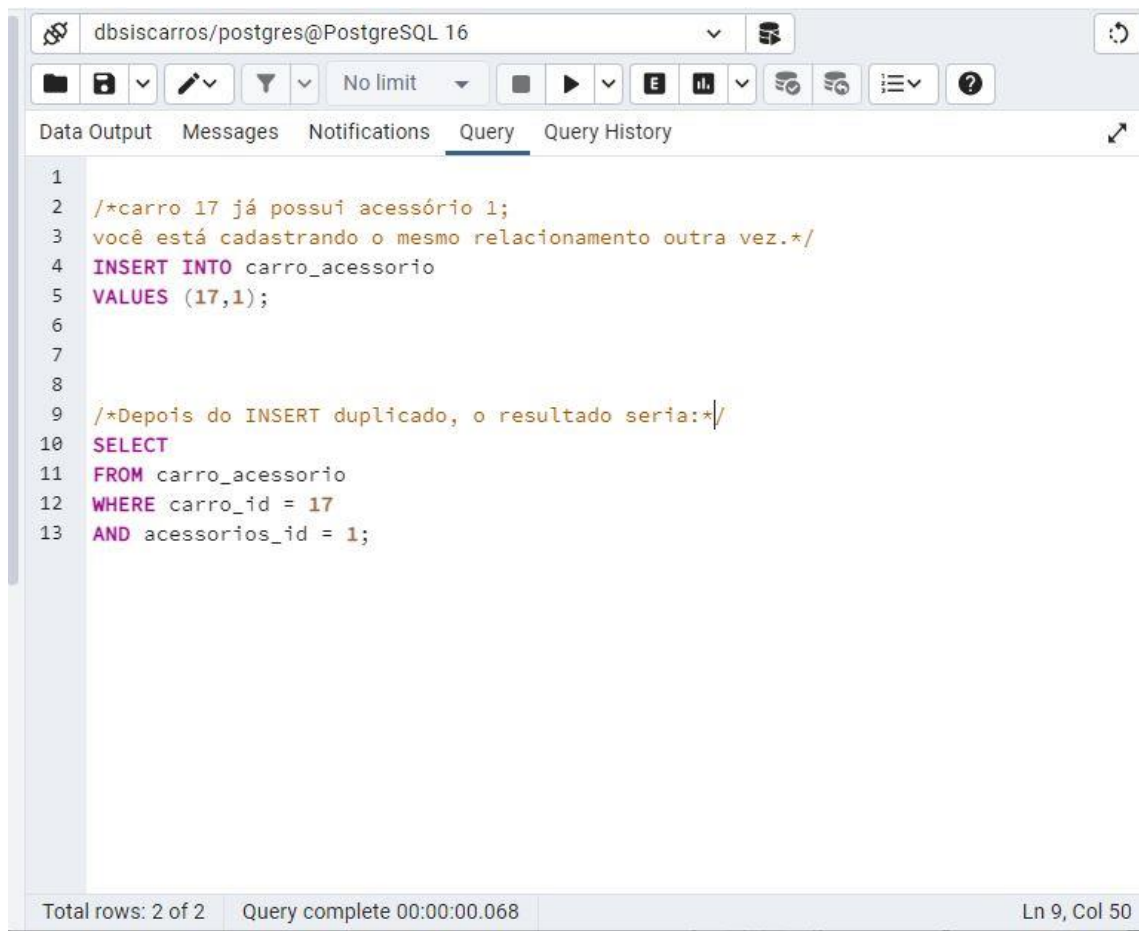


2.3.2 Ausência de chave primária composta

A tabela associativa **carro_acessorio** não possui chave primária composta nem restrição **UNIQUE** para os atributos **carro_id** e **acessorios_id**.

Como consequência, o banco permite o cadastro duplicado da mesma associação entre carro e acessório. Durante os testes realizados, verificou-se que o PostgreSQL aceita múltiplos registros idênticos da mesma combinação, comprometendo a integridade lógica dos relacionamentos.

Como solução, recomenda-se a criação de chave primária composta ou restrição **UNIQUE** para impedir duplicidades.



```
1
2  /*carro 17 já possui acessório 1;
3  você está cadastrando o mesmo relacionamento outra vez.*/
4  INSERT INTO carro_acessorio
5  VALUES (17,1);
6
7
8
9  /*Depois do INSERT duplicado, o resultado seria:*/
10 SELECT
11 FROM carro_acessorio
12 WHERE carro_id = 17
13 AND acessorios_id = 1;
```

Total rows: 2 of 2 Query complete 00:00:00.068 Ln 9, Col 50

1º comando SQL:

INSERT INTO carro_acessorio

VALUES (17,1);

- O que ele faz?

Tenta inserir novamente:

carro_id = 17

acessorios_id = 1

Só que esse registro JÁ existe na tabela.

Ou seja:

- carro 17 já possui acessório 1;
- você está cadastrando o mesmo relacionamento outra vez.

O que deveria acontecer em um banco ideal?

O banco deveria bloquear: => **ERRO: associação duplicada**

O que acontece atualmente? O PostgreSQL aceita. Porque a tabela NÃO possui:

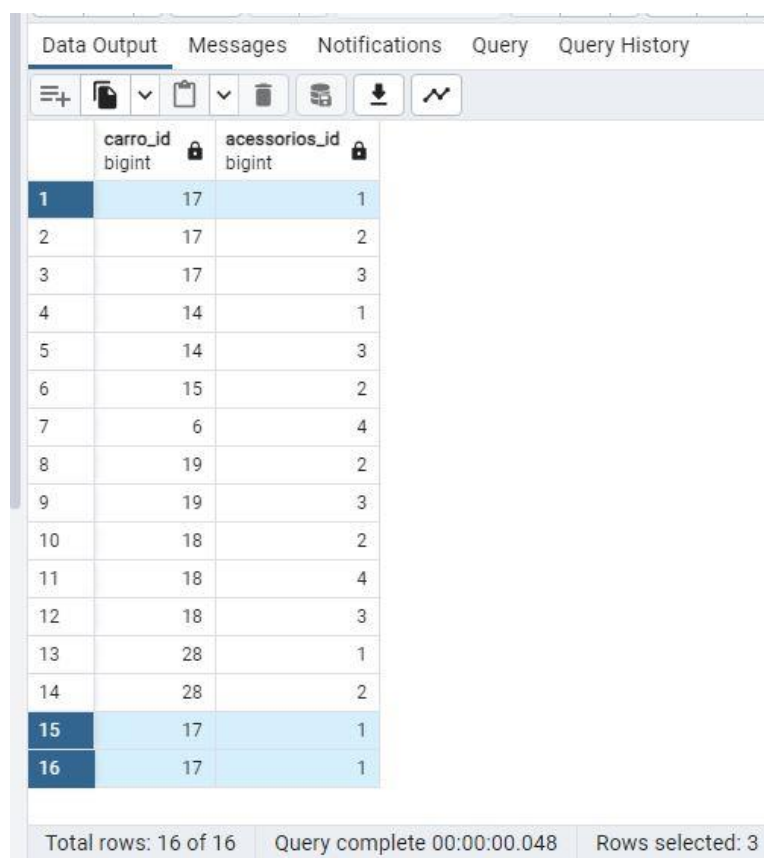
- PK composta
- UNIQUE composto

2º comando SQL:

- Mostra todos os registros, depois do INSERT duplicado, o resultado seria:

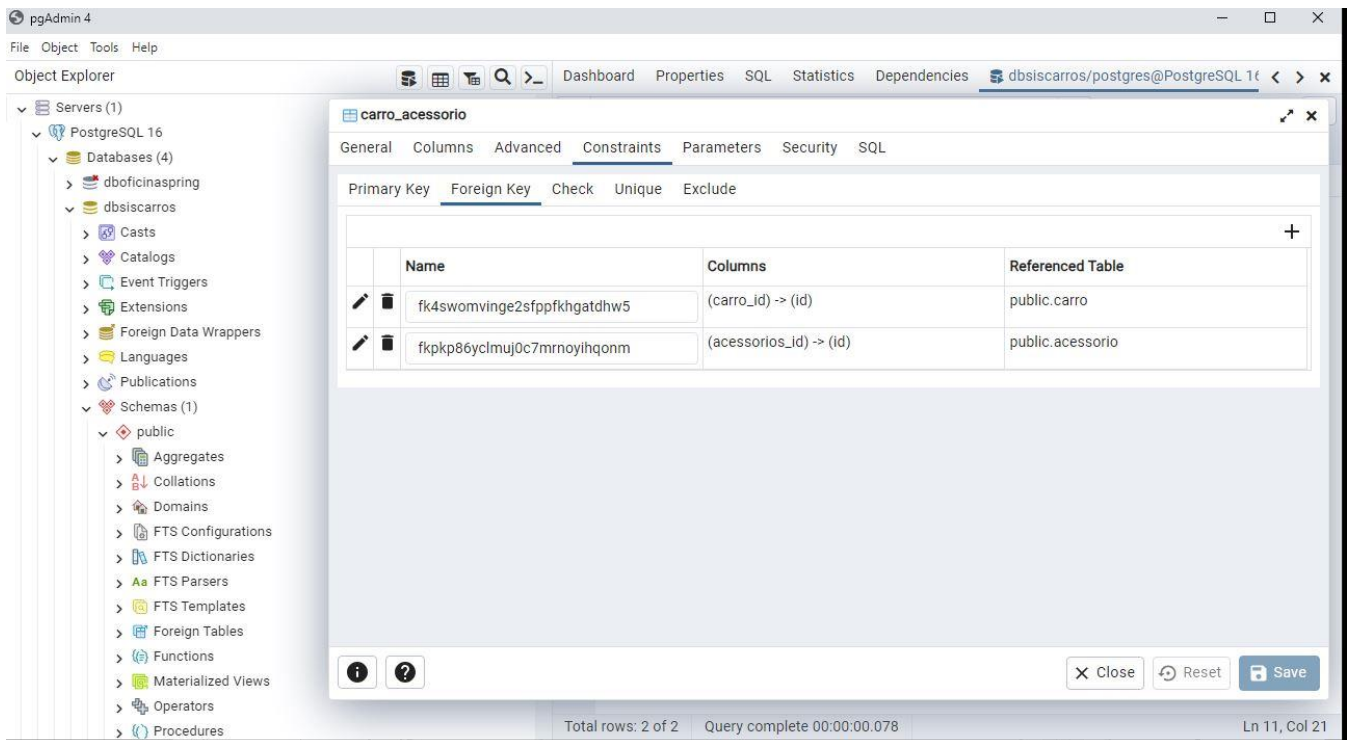
17 | 1 17 | 1

ou talvez até mais repetidos se executar várias vezes.



	carro_id bigint	acessorios_id bigint
1	17	1
2	17	2
3	17	3
4	14	1
5	14	3
6	15	2
7	6	4
8	19	2
9	19	3
10	18	2
11	18	4
12	18	3
13	28	1
14	28	2
15	17	1
16	17	1

Total rows: 16 of 16 Query complete 00:00:00.048 Rows selected: 3



A figura apresenta as constraints da tabela associativa `carro_acessorio`. Observa-se a existência de chaves estrangeiras responsáveis pela integridade referencial entre as tabelas relacionadas. Entretanto, não há definição de chave primária composta nem restrição **UNIQUE** envolvendo os campos `carro_id` e `acessorios_id`, permitindo o cadastramento de associações duplicadas entre carros e acessórios.

2.4 Normalização de Banco de Dados

2.4.1 Primeira Forma Normal (1FN)

O banco atende à Primeira Forma Normal (1FN), pois os atributos armazenam valores atômicos e não existem grupos repetitivos ou múltiplos valores em uma mesma coluna.

	id [PK] bigint	nome character varying (20)	marca_id bigint
1	5	Chevette	[null]
2	6	Golf	4
3	7	Troller	[null]
4	9	Uno	2
5	12	Lancer	[null]
6	14	FastBack	2
7	15	Punto	2
8	16	Uno Mille	2
9	17	Palio	2
10	18	Civic	3
11	19	Passat	4
12	25	Ecosport	5
13	27	Fiesta	5
14	28	Jetta	4
15	32	HRV	3
16	33	Cayenne	6

Total rows: 16 of 16 Query complete 00:00:00.153

Exemplo de tabela do banco de dados que se enquadra na 1FN.

2.4.2 Segunda Forma Normal (2FN)

A Segunda Forma Normal (2FN) é atendida pela separação das entidades em tabelas específicas, como **carro** e **marca**, evitando redundância de informações e dependências parciais entre atributos.

Dashboard Properties SQL Statistics Dependencies dbiscar

dbiscarros/postgres@PostgreSQL 16

No limit

Data Output Messages Notifications Query Query History

	id [PK] bigint	nome character varying (20)	marca_id bigint
1	5	Chevette	[null]
2	6	Golf	4
3	7	Troller	[null]
4	9	Uno	2
5	12	Lancer	[null]
6	14	FastBack	2
7	15	Punto	2
8	16	Uno Mille	2
9	17	Palio	2
10	18	Civic	3
11	19	Passat	4
12	25	Ecosport	5
13	27	Fiesta	5
14	28	Jetta	4
15	32	HRV	3
16	33	Cayenne	6

Total rows: 16 of 16 Query complete 00:00:00.153

Dashboard Properties SQL Statistics Depen

dbiscarros/postgres@PostgreSQL 16

No limit

Data Output Messages Notifications Query

	id [PK] bigint	nome character varying (20)
1	2	Fiat
2	3	Honda
3	4	Volkswagen
4	5	Ford
5	6	Porsche

Total rows: 5 of 5 Query complete 00:00:00.058

2.4.3 Terceira Forma Normal (3FN)

A Terceira Forma Normal (3FN) é identificada na modelagem do relacionamento entre carros e acessórios por meio da tabela associativa **carro_acessorio**.

Essa estrutura reduz redundâncias, evita dependências transitivas e melhora a organização do banco de dados.

Data Output Messages Notifications Query Query H			
	id [PK] bigint	nome character varying (20)	marca_id bigint
1	5	Chevette	[null]
2	6	Golf	4
3	7	Troller	[null]
4	9	Uno	2
5	12	Lancer	[null]
6	14	FastBack	2
7	15	Punto	2
8	16	Uno Mille	2
9	17	Palio	2
10	18	Civic	3
11	19	Passat	4
12	25	Ecosport	5
13	27	Fiesta	5
14	28	Jetta	4
15	32	HRV	3
16	33	Cayenne	6

Total rows: 16 of 16 Query complete 00:00:00.153

Data Output Messages Notifications		
	id [PK] bigint	nome character varying (20)
1	1	Air bag
2	2	Som
3	3	Vidro elétrico
4	4	Alarme
5	6	Aparelho MP3

Total rows: 5 of 5 Query complete 00:00:00.000

Data Output Messages Notifications		
	carro_id bigint	acessorios_id bigint
1	17	1
2	14	1
3	28	1
4	17	1
5	17	1
6	28	2
7	19	2
8	17	2
9	15	2
10	18	2
11	14	3
12	19	3
13	18	3
14	17	3
15	18	4
16	6	4

Total rows: 16 of 16 Query complete 00:00:00.000

2.5 Padrões de Segurança

2.5.1 Armazenamento inseguro de senhas

Observou-se que as senhas dos usuários estão armazenadas em texto puro no banco de dados, prática considerada inadequada do ponto de vista de segurança da informação.

Esse modelo aumenta significativamente os riscos de vazamento de credenciais em caso de acesso indevido ao banco. Recomenda-se a utilização de algoritmos de hash criptográfico, como bcrypt ou Argon2.

Data Output Messages Notifications Query Query History				
≡ + 📄 ▼ 📋 ▼ 🗑️ 🔄 📥 📤 📶 📶				
	id	password	username	
	[PK] integer	character varying (20)	character varying (10)	
1	1	admin	admin	
2	2	123456	teste	
3	3	27101995	adnogueira	
4	4	realmadrid	beckham	
Total rows: 4 of 4 Query complete 00:00:00.184				

2.5.2 Ausência de controle de acesso

O banco analisado não apresenta mecanismos explícitos de controle de acesso, como utilização de GRANT, REVOKE e definição de roles.

A utilização de privilégios específicos para diferentes perfis de usuários é fundamental para reduzir riscos de alterações indevidas e melhorar a segurança do ambiente.

2.6 Padronização de Índices

O banco de dados possui índices automáticos associados às chaves primárias. Entretanto, observou-se ausência de índices específicos em colunas utilizadas frequentemente em relacionamentos e operações **JOIN**, como **marca_id** e os atributos da tabela **carro_acessorio**.

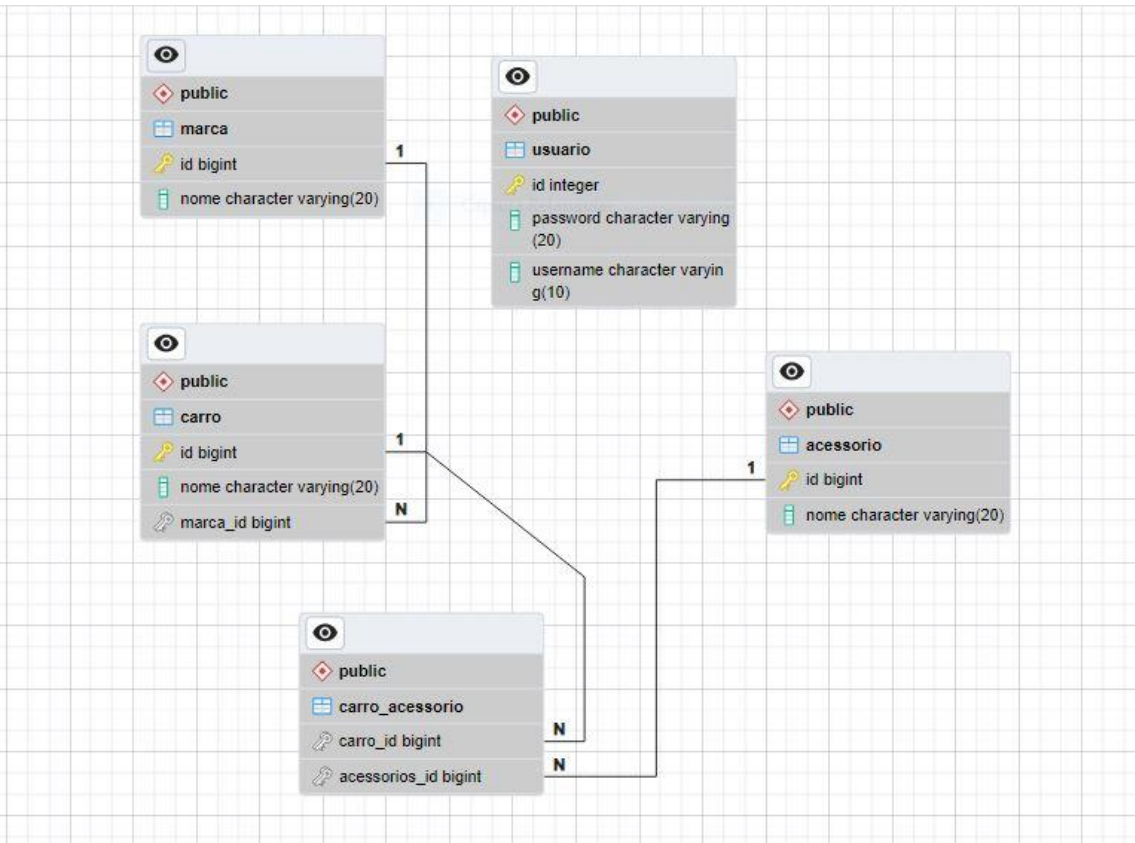
A criação estratégica de índices nessas colunas pode melhorar significativamente o desempenho das consultas realizadas pelo sistema.

Porém, deve-se evitar excesso de índices, pois isso pode impactar negativamente operações de inserção e atualização.

2.7 Documentação do Banco de Dados

2.7.1 DER e estrutura relacional

A documentação estrutural do banco pode ser realizada por meio de diagramas entidade-relacionamento (DER), permitindo melhor visualização das tabelas, atributos e relacionamentos existentes no sistema.



2.7.2 Dicionário de dados e comentários

A utilização de dicionário de dados auxilia na compreensão das responsabilidades de cada tabela do sistema.

Tabela	Descrição
carro	Armazena os veículos cadastrados
marca	Armazena as marcas dos veículos
acessorio	Armazena acessórios disponíveis

Além disso, recomenda-se a utilização de comentários técnicos diretamente no banco de dados para facilitar manutenção e governança.

3. Conclusão

A análise realizada demonstrou a importância da padronização em bancos de dados para garantir organização, integridade, desempenho e segurança das informações.

Embora o banco analisado apresente estrutura relacional adequada e atenda às formas normais de normalização, foram identificadas oportunidades de melhoria relacionadas à nomenclatura, compatibilidade de tipos de dados, integridade referencial, armazenamento seguro de senhas, controle de acesso e indexação.

A adoção das práticas propostas contribui diretamente para maior consistência estrutural, facilidade de manutenção, escalabilidade e segurança do sistema, além de promover maior qualidade no desenvolvimento de aplicações integradas ao PostgreSQL.

Link do projeto:

<https://github.com/AdrianoNog95/Trabalho-adm-banco-de-dados29-05-2026>